

Database technology II : Assignment 7 - Query optimization in Amos II

Goal

- To understand more about how a cost-based query optimizer works.
- Understanding *dynamic programming* , which is an algorithm that does an exhaustive search of the search space of execution plans, but does not bother to examine plans that it knows are slower.

Lisp

Lisp is simple. Here are the basics:

- All function calls are written as lists, which are written using parentheses. Instead of `sin(x)` to calculate the sine of x, you write `(sin x) .`
Instead of `pow(x, 3)` you write `(pow x 3) .`
Note that no commas are needed.
- All operations are function calls. Instead of `2 + 3` with the infix plus operator, imagine calling the `+` function, like this:
`+(2, 3) .`
But it's Lisp, so that call is written as a list:
`(+ 2 3) .`
Tip: You can write `(+ 2 3 4 5 6) .`
- Lisp has no operator precedence. Instead of `2 * 3 + 4 * 5 ,` which should be calculated as `(2 * 3) + (4 * 5) ,` you write `(+ (* 2 3) (* 4 5)) .`
- Also control structures like **if** are written as function calls in the form of lists: Instead of `if (a < b) f(a); else g(a, b, c);` one writes `(if (< ab) (fa) (gabc))) .`
- Variable assignments are not written with `=` but with the **setq** function . Instead of `a = 3;` one writes `(setq a 3) .`
Instead of `x = y + z + 3 * t` you write `(setq x (+ yz (* 3 t))) .`

More advanced Lisp:

- Local variables are created with **let** , which is roughly like a block in C.

```
(let (abc (d 14) (e 15) f) (setq a 17) (print a))
```

corresponds to

```
{ int a, b, c, d = 14, e = 15, f; a = 17; printf("%d\n", a); }
```

(except that the variables have no definite types in Lisp).
- In the Amos lisp you create functions with **defun** :

```
(defun xplusyz (xyz) (+ x (* yz)))
```

corresponds to

```
int xplusyz(int x, int y, int z) { return x + y * z; }
```

(except that the parameters, and the function, have no definite types).
- Note that **return** in the Amos lisp is not used to return from a function, as in C and Java, but to exit a loop. In Lisp, you only write the values to be returned, without any **return** :

```
(defun xdivy (xy) (if (equals (y 0) x (/ xy))))
```

corresponds to

```
int xdivy(int x, int y) { if ( y == 0) return x; else return x / y; }
```

Task

Write an exhaustive cost-based optimizer for Amos II. Follow [this lab instruction](#) (PDF), from the database course in Linköping. Ignore obsolete things, such as where the code skeleton "kodskelet.lsp" is located.

Code skeleton for the Lisp function **dynprogsort** : [codeskelet.lsp](#)

We have to use the older version of Amos II from [Assignment 4](#) .

For Lisp debugging to work, replace the old amos2.dmp with this one: [amos.dmp](#)

Reading tips

- The [old lab instruction](#) , from Linköping (PDF)
- [ALisp User's Guide](#) (PDF, same password as above). That is a slightly newer version than the [ALisp User's Guide](#) that is available on the web in Uppsala.
- [Amos II Release 8 User's Manual](#) (HTML)
- An [introduction to Common Lisp](#) , which is admittedly a different dialect from Alisp, but quite similar.

Accounting

Show the optimization function. Feel free to discuss with the teacher.

Work in groups of one or two students. In exceptional cases, you can work in groups of three, but ask the teacher first.

It is allowed to collaborate in larger groups than that, but each group of 1-3 students must still report separately, and it must also be clearly stated (in the report or otherwise) who participated in the collaboration.